

Plan: guardar los registros de los jugadores en Supabase

Propuesta para una próxima sesión. Todavía no hay nada programado.

1. Qué ganaríamos

- **Hoy** el progreso solo vive en el navegador (`localStorage.torreMorvath`). Si el alumno cambia de ordenador o borra el navegador, lo pierde. El profesor solo ve lo que cada alumno le descarga con el informe.
- **Con Supabase:**
 - el progreso sigue al alumno de un ordenador a otro;
 - el profesor tiene un panel con toda la clase: pisos superados, precisión por criterio y quién se ha atascado dónde.

2. Coste y límites (plan gratuito)

- **Gratis:** 500 MB de base de datos y hasta 50.000 usuarios activos al mes. Para unas clases sobra.
- **Pausa por inactividad:** si el proyecto pasa una semana sin uso, Supabase lo pausa y hay que reactivarlo a mano desde el panel. Durante las vacaciones puede pasar.
- **Sin tarjeta:** no hace falta tarjeta para el plan gratuito.

3. Privacidad (lo más importante)

Son alumnos menores en un instituto de Canarias, así que se aplica el RGPD y la LOPDGDD. La regla es **guardar lo mínimo**:

- Nada de correos, teléfonos ni nombres completos de los alumnos. Basta con un **alias** (por ejemplo «Leo-3B») y el **código de la clase**.
- **Qué se guarda:** pisos superados, aciertos y fallos por criterio y tiempos.
 - No se guarda el texto que escriben (contraseñas de prueba, HTML...).
 - Tampoco se guarda el chat.
- **Quién decide:** el centro educativo es quien decide si se usa. Conviene que el profesor lo consulte con el equipo directivo o con el delegado de protección de datos antes de activarlo.
- **Aviso en el juego:** que diga qué se guarda y para qué.

4. Diseño

Tablas

Tabla	Campos	Quién escribe	Quién lee
clases	id, codigo (6 letras), nombre, profesor_id	El profesor, desde el panel	El profesor de esa clase
alumnos	id, clase_id, alias, creado	El juego, al unirse con el código de clase	El profesor de esa clase
progreso	alumno_id, piso, completado, mejor_precision, segundos, actualizado	El juego, al superar un piso	El alumno (el suyo) y el profesor
criterios	alumno_id, criterio, aciertos, fallos	El juego	El alumno y el profesor

Seguridad

- **Seguridad por filas (RLS)** activada en todas las tablas.
- **La clave pública ("anon")** puede ir en el código del juego, porque las políticas RLS limitan lo que permite hacer.
- **El alumno:**
 - entra con el código de clase y su alias, sin cuenta ni correo;
 - Supabase le da una sesión anónima («anonymous sign-in»);
 - solo puede leer y escribir **sus** filas.
- **El profesor:**
 - inicia sesión con su correo (enlace mágico);
 - solo ve las clases que ha creado.

En el juego

1. En la portada aparece un botón «Unirme a mi clase»: código de clase y alias.
2. `guardarProgreso()` sigue guardando en el navegador como hoy y, además, encola el cambio para Supabase.
3. Si no hay internet, la cola espera en `localStorage` y se envía después. El juego nunca depende de la base de datos para funcionar.
4. Al entrar con el mismo alias y código desde otro ordenador, se recupera el progreso.
5. **Librería:** `@supabase/supabase-js` por CDN (jsDelivr, como PeerJS). No hace falta compilar nada.

Panel del profesor

- Una página aparte, `profesor.html`, con su inicio de sesión. Permite:
 - crear clases;
 - ver la tabla de alumnos por piso y por criterio;
 - descargar el informe de toda la clase (reutilizando `informe.js`).

5. Qué tendría que hacer el usuario

Claude no puede crear cuentas ni meter credenciales en formularios web.

1. Crear una cuenta gratuita en supabase.com y un proyecto (región Europa, por ejemplo Fráncfort).
2. Activar «Anonymous sign-ins» en *Authentication* → *Providers*.
3. Pasar a Claude la **URL del proyecto** y la **clave pública anon**. La clave `service_role` **nunca**: esa no va en el juego ni en el repositorio.
4. Claude prepararía:
 - el SQL de las tablas y las políticas RLS, para pegarlo en el *SQL Editor*;
 - el código del juego;
 - el panel del profesor;
 - las pruebas.

6. Orden de trabajo propuesto (una sesión larga)

1. Escribir el SQL de las tablas y las políticas, y probar las políticas con dos alumnos falsos.
2. Crear `js/nube.js`: unirse a la clase, la cola de envíos y la recuperación del progreso, con pruebas de la cola en Node.
3. Añadir el botón y el aviso de privacidad en la portada.
4. Hacer `profesor.html` con la tabla de la clase y el informe.
5. Hacer una prueba completa con dos navegadores (alumno y profesor) y documentarla en el plan técnico.